SERVICIOS WEB Y BASES DE DATOS NO RELACIONALES

DESARROLLO DE SOFTWARE Y ARQUITECTURA CLOUD

NODE.JS • EXPRESS • MONGODB ATLAS • VERCEL

# REPORTE TÉCNICO Y DOCUMENTACIÓN: SERVICIO API REST PARA COLECCIÓN "EMPLEADOS"

Implementación de operaciones CRUD (POST, GET, PUT, DELETE), conexión a clúster NoSQL MongoDB Atlas con almacenamiento de tipos BSON Date y publicación continua en la nube con Vercel.

AUTOR / DESARROLLADOR

Jesús Zamora

BASE DE DATOS

MongoDB Atlas (NoSQL DB: miBD)

PLATAFORMA DE HOSTING

Vercel (Serverless Functions)

REPOSITORIO GITHUB

github.com/Jesuszruelas/webAPI

## 1. Información del Hosting y Enlaces en la Nube

El servicio se encuentra publicado de forma continua y funcional en la nube a través de **Vercel** como arquitectura Serverless en Node.js, conectado a un clúster de base de datos No Relacional en **MongoDB Atlas**.

| | |
|---|---|
| **Nombre del Hosting:** | **Vercel** (Vercel Serverless Architecture) |
| **URL Base en Producción:** | `https://miapi-pi.vercel.app` |
| **URL Endpoint Principal:** | `https://miapi-pi.vercel.app/api/empleados` |
| **URL Alternativa Directa:** | `https://miapi-pi.vercel.app/Empleados` |
| **Repositorio de Código:** | `https://github.com/Jesuszruelas/webAPI` |
| **Motor de Base de Datos:** | **MongoDB Atlas** (Base de datos: `miBD`) |
| **Nombre de la Colección:** | `Empleados` (No-Relational Collection) |

## 2. Especificación de Campos de la Colección "Empleados"

Al tratarse de una base de datos No Relacional, el esquema almacena los campos requeridos y permite flexibilidad en la inserción de documentos:

| CAMPO | TIPO REQUERIDO | TIPO EN MONGODB / BSON | OBLIGATORIO | DESCRIPCIÓN |
|---|---|---|---|---|
| **Clave** | Numérica | `Number / Double` | Sí | Clave numérica identificadora del empleado. |
| **Nombre** | Texto | `String` | Sí | Nombre completo del empleado. |
| **dirección** | Texto | `String` | Sí | Dirección residencial (soporta alias `direccion`). |
| **fechadenacimiento** | BSON Date | `Date (ISODate)` | Sí | Fecha de nacimiento persistida nativamente como BSON Date. |
| **sexo** | Numérico o Texto | `Mixed (String/Number)` | Sí | Acepta texto ("M", "F", "Masculino") o código numérico (1, 2). |
| **tipodesangre** | Texto | `String` | Sí | Tipo sanguíneo y factor Rh (ej: "O+", "A+"). |

## 3. Catálogo de Operaciones REST

| MÉTODO | URL EN LA NUBE (VERCEL) | DESCRIPCIÓN DE LA OPERACIÓN |
|---|---|---|
| `GET` | `https://miapi-pi.vercel.app/` | Verifica estado del servicio y lista las rutas disponibles. |
| `GET` | `https://miapi-pi.vercel.app/api/empleados` | Obtiene la lista de todos los empleados de la base de datos. |

| MÉTODO | URL EN LA NUBE (VERCEL) | DESCRIPCIÓN DE LA OPERACIÓN |
| --- | --- | --- |
| GET | `https://miapi-pi.vercel.app/api/empleados/:clave` | Consulta un empleado específico por su Clave numérica o su _id. |
| POST | `https://miapi-pi.vercel.app/api/empleados` | Inserta un nuevo empleado en la colección "Empleados". |
| PUT | `https://miapi-pi.vercel.app/api/empleados/:clave` | Actualiza los datos de un empleado identificado por su Clave o _id. |
| DELETE | `https://miapi-pi.vercel.app/api/empleados/:clave` | Elimina un empleado de la colección por su Clave o _id. |

## 4. Ejemplos de RAW JSON para Pruebas en POSTMAN

Configuración general: Header `Content-Type: application/json`.

### A) Operación POST — Registrar Nuevo Empleado

**URL:** `https://miapi-pi.vercel.app/api/empleados`

PAYLOAD RAW JSON DE ENTRADA:

```
{
  "Clave": 101,
  "Nombre": "Carlos Mendoza López",
  "dirección": "Av. Revolución 123, Col. Centro",
  "fechadenacimiento": "1992-05-14",
  "sexo": "M",
  "tipodesangre": "O+"
}
```

RESPUESTA REAL DE MONGODB ATLAS (HTTP 201 CREATED):

```
{
  "_id": "6adaadd6af9d40e140a41468",
  "Clave": 101,
  "Nombre": "Carlos Mendoza López",
  "dirección": "Av. Revolución 123, Col. Centro",
  "fechadenacimiento": "1992-05-14T00:00:00.000Z",
  "sexo": "M",
  "tipodesangre": "O+",
  "createdAt": "2026-10-23T00:44:06.600Z",
  "updatedAt": "2026-10-23T00:44:06.600Z"
}
```

### B) Operación GET — Listar Todos los Empleados

**URL:** `https://miapi-pi.vercel.app/api/empleados` (o `https://miapi-pi.vercel.app/Empleados`)

RESPUESTA REAL DE LA API (HTTP 200 OK):

```
[
  {
    "_id": "6adaadd6af9d40e140a41468",
    "Clave": 101,
    "Nombre": "Carlos Mendoza López",
    "dirección": "Av. Revolución 123, Col. Centro",
    "fechadenacimiento": "1992-05-14T00:00:00.000Z",
    "sexo": "M",
    "tipodesangre": "O+",
    "createdAt": "2026-10-23T00:44:06.600Z",
    "updatedAt": "2026-10-23T00:44:06.600Z"
  },
  {
    "_id": "6adaafc2a0f70b61df944404",
    "Clave": 101,
    "Nombre": "Empleado Con Misma Clave",
    "dirección": "Segunda Dirección",
    "fechadenacimiento": "1995-01-01T00:00:00.000Z",
    "sexo": "F",
    "tipodesangre": "AB+",
    "createdAt": "2026-10-23T00:52:18.360Z",
    "updatedAt": "2026-10-23T00:52:18.360Z"
```

```
  }
]
```


## C) Operación GET — Consultar Empleado por Clave

**URL:** `https://miapi-pi.vercel.app/api/empleados/101`

RESPUESTA REAL DE LA API (HTTP 200 OK):


```
{
  "_id": "6adaadd6af9d40e140a41468",
  "Clave": 101,
  "Nombre": "Carlos Mendoza López",
  "dirección": "Av. Revolución 123, Col. Centro",
  "fechadenacimiento": "1992-05-14T00:00:00.000Z",
  "sexo": "M",
  "tipodesangre": "O+"
}
```

### D) Operación PUT — Actualizar Datos de Empleado

**URL:** `https://miapi-pi.vercel.app/api/empleados/101`

PAYLOAD RAW JSON DE ENTRADA:

```
{
  "Nombre": "Carlos Mendoza López (Actualizado)",
  "dirección": "Blvd. Paseo de los Héroes #1200, Zona Río",
  "tipodesangre": "O-"
}
```

RESPUESTA REAL DE LA API (HTTP 200 OK):


```
{
  "_id": "6adaadd6af9d40e140a41468",
  "Clave": 101,
  "Nombre": "Carlos Mendoza López (Actualizado)",
  "dirección": "Blvd. Paseo de los Héroes #1200, Zona Río",
  "fechadenacimiento": "1992-05-14T00:00:00.000Z",
  "sexo": "M",
  "tipodesangre": "O-",
  "updatedAt": "2026-10-23T00:54:00.000Z"
}
```


### E) Operación DELETE — Eliminar Empleado

**URL:** `https://miapi-pi.vercel.app/api/empleados/101`

RESPUESTA REAL DE LA API (HTTP 200 OK):


```
{
  "mensaje": "Empleado eliminado",
  "empleado": {
    "_id": "6adaadd6af9d40e140a41468",
    "Clave": 101,
    "Nombre": "Carlos Mendoza López (Actualizado)"
  }
}
```


## 5. Código del Servicio (Archivos de Configuración)

VERCEL.JSON

```
{
  "version": 2,
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/api/index.js"
    }
  ]
}
```

API/INDEX.JS

```
const app = require('../index');

module.exports = app;
```

## 6. Código del Servicio Principal (index.js)

INDEX.JS — IMPLEMENTACIÓN BACKEND EXPRESS + MONGOOSE

```
require('dotenv').config();
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');

const app = express();
app.set('json spaces', 2);
app.use(cors());
app.use(express.json());

// Normalización de rutas para aceptar /api/empleados, /Empleados y /empleados
app.use((req, res, next) => {
    if (req.url.endsWith(':')) req.url = req.url.slice(0, -1);
    const lowerUrl = req.url.toLowerCase();
    if (lowerUrl.startsWith('/empleados')) req.url = '/api' + lowerUrl;
    else if (lowerUrl.startsWith('/api/empleados')) req.url = lowerUrl;
    next();
});

let cachedDb = null;
async function connectToDatabase() {
    if (cachedDb && mongoose.connection.readyState === 1) return cachedDb;
    const mongoUri = process.env.MONGO_URI;
    if (!mongoUri) throw new Error('MONGO_URI no configurado.');
    cachedDb = await mongoose.connect(mongoUri, { serverSelectionTimeoutMS: 5000 });
    return cachedDb;
}

const empleadoSchema = new mongoose.Schema({
    Clave: { type: Number, required: [true, 'El campo Clave es obligatorio'] },
    Nombre: { type: String, required: [true, 'El campo Nombre es obligatorio'], trim: true },
    dirección: { type: String, required: [true, 'El campo dirección es obligatorio'], trim: true, alias:
'direccion' },
    fechadenacimiento: { type: Date, required: [true, 'El campo fechadenacimiento es obligatorio'] },
    sexo: { type: mongoose.Schema.Types.Mixed, required: [true, 'El campo sexo es obligatorio'] },
    tipodesangre: { type: String, required: [true, 'El campo tipodesangre es obligatorio'], trim: true }
}, { timestamps: true, versionKey: false });

const Empleado = mongoose.models.Empleado || mongoose.model('Empleado', empleadoSchema, 'Empleados');

function normalizarEmpleado(data) {
    const n = {};
    if (data.Clave !== undefined || data.clave !== undefined) n.Clave = Number(data.Clave ?? data.clave);
    if (data.Nombre !== undefined || data.nombre !== undefined) n.Nombre = String(data.Nombre ??
data.nombre).trim();
    if (data.dirección !== undefined || data.direccion !== undefined) n.dirección = String(data.dirección ??
data.direccion).trim();
    if (data.fechadenacimiento || data.fechaDeNacimiento || data.fecha_nacimiento) {
        n.fechadenacimiento = new Date(data.fechadenacimiento || data.fechaDeNacimiento ||
data.fecha_nacimiento);
    }
    if (data.sexo !== undefined || data.Sexo !== undefined) n.sexo = data.sexo ?? data.Sexo;
    if (data.tipodesangre || data.tipoDeSangre || data.tipo_sangre) {
        n.tipodesangre = String(data.tipodesangre || data.tipoDeSangre || data.tipo_sangre).trim();
    }
    return n;
}

const requireDB = async (req, res, next) => {
```

```
    try { await connectToDatabase(); next(); }
    catch (error) { res.status(500).json({ error: 'Error de conexión con MongoDB Atlas', detalle: error.message
}); }
};

app.get('/', (req, res) => {
    res.json({ servicio: 'API REST MongoDB Atlas - Empleados', estado: 'En línea', rutas: ['/api/empleados'] });
});

app.get('/api/empleados', requireDB, async (req, res) => {
    try { const emp = await Empleado.find().sort({ Clave: 1 }); res.status(200).json(emp); }
    catch (error) { res.status(500).json({ error: error.message }); }
});

app.get('/api/empleados/:identificador', requireDB, async (req, res) => {
    try {
        const { identificador } = req.params;
        const filtro = !isNaN(identificador) ? { Clave: Number(identificador) } : { _id: identificador };
        const emp = await Empleado.findOne(filtro);
        if (!emp) return res.status(404).json({ error: 'Empleado no encontrado' });
        res.status(200).json(emp);
    } catch (error) { res.status(500).json({ error: error.message }); }
});

app.post('/api/empleados', requireDB, async (req, res) => {
    try {
        const datos = normalizarEmpleado(req.body);
        if (datos.Clave === undefined || isNaN(datos.Clave)) return res.status(400).json({ error: 'Clave
obligatoria y numérica' });
        if (!datos.Nombre) return res.status(400).json({ error: 'Nombre obligatorio' });
        if (!datos.dirección) return res.status(400).json({ error: 'dirección obligatoria' });
        if (!datos.fechadenacimiento || isNaN(datos.fechadenacimiento.getTime())) return res.status(400).json({
error: 'fechadenacimiento obligatoria' });
        if (datos.sexo === undefined || datos.sexo === null) return res.status(400).json({ error: 'sexo
obligatorio' });
        if (!datos.tipodesangre) return res.status(400).json({ error: 'tipodesangre obligatorio' });

        const nuevo = new Empleado(datos);
        const guardado = await nuevo.save();
        res.status(201).json(guardado);
    } catch (error) { res.status(400).json({ error: error.message }); }
});

app.put('/api/empleados/:identificador', requireDB, async (req, res) => {
    try {
        const { identificador } = req.params;
        const filtro = !isNaN(identificador) ? { Clave: Number(identificador) } : { _id: identificador };
        const datos = normalizarEmpleado(req.body);
        const act = await Empleado.findOneAndUpdate(filtro, { $set: datos }, { returnDocument: 'after',
runValidators: true });
        if (!act) return res.status(404).json({ error: 'Empleado no encontrado' });
        res.status(200).json(act);
    } catch (error) { res.status(400).json({ error: error.message }); }
});

app.delete('/api/empleados/:identificador', requireDB, async (req, res) => {
    try {
        const { identificador } = req.params;
        const filtro = !isNaN(identificador) ? { Clave: Number(identificador) } : { _id: identificador };
        const del = await Empleado.findOneAndDelete(filtro);
        if (!del) return res.status(404).json({ error: 'Empleado no encontrado' });
        res.status(200).json({ mensaje: 'Empleado eliminado', empleado: del });
    } catch (error) { res.status(500).json({ error: error.message }); }
});

const PORT = process.env.PORT || 3000;
```

```
if (process.env.NODE_ENV !== 'production' || require.main === module) {
    app.listen(PORT, () => console.log(`Servidor en http://localhost:${PORT}`));
}
module.exports = app;
```